PRECOP: The Predictive Covenant Oracle Protocol

Yellowpaper: Sovereign Conditional State Machines and Thermodynamic Consensus on Bitcoin L1

laz1m0v (Lead Protocol Architect & Cryptography Engineer)

March 2026

Version 0.0.1

Abstract

The decentralization of complex state machines on the Bitcoin base layer (L1) has historically been hindered by the absence of Turing completeness and an inherent reliance on trusted oracles. The PRECOP (Predictive Covenant Oracle Protocol) addresses this asymmetry by introducing a strict cryptographic paradigm where the Taproot output key is a deterministic commitment to the system's state vector. By leveraging Taproot primitives (BIP-341, BIP-342) and Simplicity covenants via the **Astrolabe** pattern, PRECOP enables the instantiation of Sovereign Conditional State Machines—encompassing Prediction Markets, Automated Market Makers (AMMs), and Collateralized Synthetic Assets—directly onto the UTXO set. This paper formalizes the protocol's architecture, the deterministic serialization of state, the thermodynamic proof-of-work (Binohash), and the underlying mathematical invariants guaranteeing a robust Nash Equilibrium while substantially reducing discretionary trust.

"Sovereign conditional states where the covenant is the consensus."

1 Introduction & Architectural Philosophy

1.1 The Oracle Paradox and the Necessity of Thermodynamic Consensus

Within the ecosystem of cryptographic protocols, the "oracle problem" denotes the inability of a blockchain to autonomously and deterministically verify exogenous information. Historically, dominant architectures have mitigated this deficiency by introducing trusted federations (multi-sig quorums) or secondary consensus layers (Proof-of-Stake altchains). These models reintroduce vulnerabilities to Byzantine faults (BFT) and establish an asymmetry of trust that contradicts the fundamental axiom of Bitcoin: *Don't trust, verify.*

PRECOP redefines truth acquisition by subjecting it to a **Thermodynamic Consensus**. Inspired by foundational decentralized oracle theories [9], the resolution of an external condition is no longer a declarative act or a delegated vote; it is a cryptographic proof backed by tangible energy expenditure (Proof-of-Work). The protocol transmutes "trust in a third party" into "trust in thermodynamics and game theory."

1.2 Axiomatic Extension: Conditional State Machines

PRECOP's mathematical formalism facilitates the on-chain execution of Automated Market Makers (AMM) governing constant product liquidity curves ($x \cdot y = k$), as well as the management of sovereign collateral (the BTCDAI algorithmic stablecoin). The *Covenant*—the smart contract encoded within the Merkle Abstract Syntax Tree (MAST)—replaces discretionary trust with explicit cryptographic constraints, drawing on the theoretical framework of self-reproducing coins in UTXO models [12], acting as the arbiter of state transitions. This formalism also supports native Runes-compatible tokens via the same covenant structure (future extension).

1.3 Strictly Isolated 6-Layer Architecture

To guarantee absolute determinism, PRECOP imposes strict domain separation. No network anomaly or asynchronous I/O operation can alter the state machine. Let $\mathcal{L}$ be the set of the protocol's layers:

- **Layer 0 (On-Chain Consensus):** The enforcement layer. Composed of Simplicity programs (6-leaf MAST topology). It unconditionally rejects any transaction violating mathematical invariants.
- **Layer 1 (Cryptographic Core):** The pure mathematics library (hashing functions, secp256k1 derivation, LMSR pricing). This layer formally proscribes the use of floating-point arithmetic.
- **Layer 2 (Transactional Construction):** The orchestration of Witnesses and Taproot derivations. Handles the transition from an OPI-2 intent to a valid Bitcoin transaction.
- **Layer 3 (IndexerClaw & LSVM):** The Local Simplicity Verification Module, a localized virtual machine replicating Layer 0 consensus. It natively handles chain reorganizations (reorgs) and enforces the `ConsensusGuardError`.
- **Layer 4 (Runtime & Relay):** The transmission interface broadcasting transactions to the P2P network (Bitcoin L1 / Liquid).
- **Layer 5 (DONCLAW Oracle Agents):** The decentralized network of autonomous entities executing heuristic inference (LLMs) coupled with cryptographic extraction (Binohash PoW).

1.4 Security Assumptions

The protocol's security properties rely on the following axiomatic assumptions:

1. **Oracle Honesty**: A majority of oracle agents (weighted by thermodynamic effort) remain economically honest.
2. **Computational Cost**: The **Binohash** grinding cost ($W = 42$) remains non-trivial and resistant to sudden hardware-driven difficulty shifts.
3. **Determinism**: The **LSVM** implementations are bug-free and strictly consensus-compatible with the target environment.
4. **Integrity**: Taproot/Simplicity compilation artifacts used by verifiers match the source commitments defined in this paper.
5. **Entropy**: The **UTXOracle** price extraction remains sufficiently entropic; i.e., target Bitcoin blocks contain enough diverse payment-representative transactions.
6. **Policy Compliance**: Resolver nodes adhere to the batch inclusion policy, preventing adversarial divergence in intent ordering beyond protocol-level tolerance.

2 Sovereign State Model: Tapdata & BIP-341

The cornerstone of PRECOP resides in the **Astrolabe (Tapdata)** pattern. Building upon state-commitment paradigms introduced by protocols like RGB [7] and Taproot Assets [8], PRECOP encodes the state as a scalar modifier of the UTXO's public key. Thus, **the Taproot output key is a deterministic commitment to the state vector [1]**.

2.1 Topology of the State Vector ($\vec{S}_{L1}$)

To maximize efficiency on the Bit Machine, the market state is hyper-compressed into a 31-byte vector committed within the Taproot tweak.

Offset	Size	Field	Definition & Type
0x00	1 byte	`version`	Protocol version (0x05)
0x01	8 bytes	`market_id`	Truncated SHA-256 prefix: uint64 BE
0x09	1 byte	`phase`	Phase (1=OPEN, 2=VOTING, 3=RESOLVED)
0x0A	8 bytes	`q_yes`	YES share quantity: uint64 BE
0x12	8 bytes	`q_no`	NO share quantity: uint64 BE
0x1A	4 bytes	`oracle_cnt`	Thermodynamic proof counter: uint32 BE
0x1E	1 byte	`resolution`	Outcome (0=PENDING, 1=YES, 2=NO)

Table 1: Topology of the State Vector

2.2 The BTCDAI Synthetic State Machine (31 bytes)

The state of a collateralization vault for the algorithmic stablecoin.

2.3 Deterministic State Mutation: Real Mutinynet Examples

Each valid state spend MUST pay to the next address in the sequence, enforced by the Simplicity `jet::build_taptweak`.

Offset	Size	Field	Definition & Type
0x00	1 byte	`version`	Protocol version (0x05)
0x01	8 bytes	`vault_id`	Unique vault identifier: uint64 BE
0x09	1 byte	`phase`	Phase (1=OPEN, 2=LIQUIDATED)
0x0A	8 bytes	`collateral`	Locked Satoshis: uint64 BE
0x12	8 bytes	`debt`	Issued BTCDAI units: uint64 BE
0x1A	5 bytes	`reserved`	Zero-padded padding (u40)

Table 2: BTCDAI Synthetic State Machine

Step	State Vector Summary	Resulting P2TR Address
DEPLOY	`version=05, phase=OPEN, q_yes=0, q_no=0`	`tb1pqrap...g9n`
BUY 100 YES	`version=05, phase=OPEN, q_yes=100, q_no=0`	`tb1plx6k...jn6`
RESOLVED (YES)	`version=05, phase=RESOLVED, q_yes=100, q_no=0, resolution=1, oracle_cnt=42`	`tb1pzn9w...taj`

Table 3: State Transition Examples

2.4 Taproot Key Derivation (Q) and the Transition Invariant

Let P be the internal **NUMS** key (nothing-up-my-sleeve) defined as:

$$P = \texttt{0x50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0}$$

The final output public key Q_t for state $\vec{S}_t$ is tweaked by the Merkle Root M_t of the 6-leaf MAST:

$$Q_t = P + \text{tagged_hash}(\text{TapTweak}, P \parallel M_t)G$$

The Axiomatic Transition Theorem: A state transition is valid if and only if the Simplicity Covenant verifies that $\vec{S}_{t+1} = \Upsilon(\vec{S}_t, I_t, W_t)$ and that the output UTXO is locked under Q_{next} (derived from $\vec{S}_{t+1}$), where:

- $\vec{S}_t$ is the current state vector.
- I_t is the canonical intent or operation payload (OPI-2).
- W_t is the witness or hint set (e.g., pricing ZKP-hints) required by the covenant logic.

In practice, this is enforced by the Simplicity instruction `jet::build_taptweak` which recomputes the new commitment from the mutated state vector before allowing the spend.

3 Covenant Topology: The 6-Leaf MAST

To remain within the computational bounds of the Bit Machine while supporting complex AMM and stablecoin logic, PRECOP decomposes its validation rules into a 6-leaf **Merkle Abstract Syntax Tree (MAST)** [10].

3.1 Cryptographic Identity (CMR)

The commitment to the protocol's rules is defined by the **Commitment Merkle Root (CMR)** of the Simplicity programs. These hashes are the immutable law of the protocol.

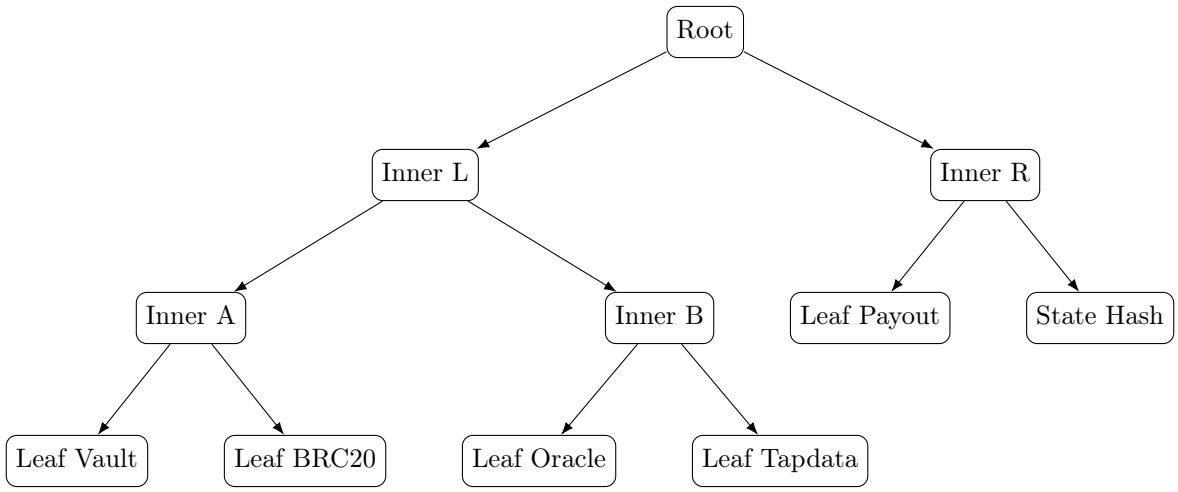
Global MAST Root: `3225331c38dccf5707be0e07...`

Contract	Role	CMR (Identity)	TapLeaf Hash	Status
`vault_binohash.simf`	**Main Router**	`8a4d0163...`	`0550a41e...`	Compiled
`brc20_token.simf`	**Token Sentinel**	`f99b37f7...`	`2b48008e...`	Compiled
`oracle_validation.simf`	**Truth Mining**	`3debe...`	`bce51d4b...`	Compiled
`tapdata_state.simf`	**State Invariant**	`1c36b...`	`565708f1...`	Compiled
`payout_enforcement.simf`	**Distribution**	`39ff3...`	`b80d9a46...`	Compiled
`w_token.simf`	**W Token Covenant**	`84d676...`	`97ff122c...`	Compiled

Table 4: MAST Leaf Commitments

3.2 MAST Topology

The root M is derived from the following hierarchy:



This topology ensures that verifying a simple trade (Path: Vault + Tapdata) is computationally cheaper than verifying a market resolution (Path: Payout + Oracle).

4 OPI Framework: Transcending the Intent Standards

PRECOP does not merely adopt the OPI-1 and OPI-2 standards; it **transcends** them. While these standards define a logical framework for off-chain interpretation, PRECOP compiles their semantic rules — swap logic, liquidity locks, and emission schedules — directly into the Simplicity MAST (Layer 0). The covenant becomes the ultimate, deterministic executor of the OPI constitution: every state transition is validated at the consensus level, making the UTXO set itself the canonical, sovereign state machine.

4.1 Transport vs. Protocol — An Explicit Distinction

PRECOP is **not** a BRC-20 protocol. It is an autonomous Simplicity-covenant state machine that happens to use the BRC-20 JSON envelope `{"p":"brc-20",...}` as a **broadcast transport**.

Layer	Authority	Who decides validity?
JSON v3 envelope	**None**	Indexer routing only
IndexerClaw (LSVM)	**Structural**	Rejects malformed intents
Simplicity Covenant	**Absolute**	The only arbiter of state truth

Table 5: Authority Hierarchy

4.2 PRECOP Opcode Map

The following tables define the complete binary opcode space. Total: **20 registered processors**.

Opcode	Operation	OPI	Size	Structure
`0x01`	DEPLOY	Internal	14	`[PREC] [0x05] [0x01] [tick4] [params]`
`0x02`	BUY	Internal	15	`[PREC] [0x05] [0x02] [tick4] [side] [qty]`
`0x10`	VOTE	Internal	38	`[PREC] [0x05] [0x10] [tick4] [outcome] [nonce]`
`0x51`	BTCDAI_MINT	Internal	30	`[PREC] [0x05] [0x51] [col] [debt] [price]`
`0x60`	SWAP_INIT	OPI-1	22	`[PREC] [0x05] [0x60] [tickA] [tickB] [amt] [lock]`

Table 6: Selected Opcodes (Non-exhaustive)

4.3 Formal Specification of Canonical Batching (CBM)

To resolve the "Hot UTXO" contention bottleneck, PRECOP implements a **Canonical Batch Model (CBM)**. By consolidating k intents into a single L1 transition, PRECOP reduces the settlement footprint and ensures that the system handles transaction volume at the throughput limit of the Bit Machine.

5 Financial Mathematics & Invariants

5.1 Deterministic LMSR Pricing

The LMSR cost increment Δ_{cost} for acquiring Δ_{shares} is verified using ZKP-Hints $E_y = \exp(q_y/b)$ and $E_n = \exp(q_n/b)$. The covenant enforces:

$$\Delta_{cost} \cdot (E_y + E_n) \approx E_y \cdot S \pm \epsilon$$

5.2 OPI-1 AMM Invariants

The swap execution layer implements the constant-product AMM invariant ($x \cdot y = k$).

$$\Delta_y \cdot (R_x + \Delta_x) \approx R_y \cdot \Delta_x \pm \epsilon$$

5.3 BTCDAI Collateralization Invariants

- **Minting**: Collateral $\times$ Price$_{btc}$ $\geq$ Debt $\times$ 1.5×10^8.
- **Liquidation**: Collateral $\times$ Price$_{btc}$ $<$ Debt $\times$ 1.2×10^8.

5.4 UTXOracle v9.1: Trustless Price Extraction

PRECOP resolves the collateralization of BTCDAI using **UTXOracle v9.1**, a 12-step deterministic algorithm that extracts the BTC/USD price directly from on-chain transaction patterns.

6 Conclusion

PRECOP v0.0.1 proposes a unified synthesis of covenant-enforced state transitions, thermodynamic oracle selection, and compact Bitcoin-native intent encoding. By committing state into the Taproot output key and verifying invariants through Simplicity MAST, it substantially reduces discretionary trust by replacing committee-level discretion with cryptographic commitments and economically constrained resolution.

References

[1] Wuille, P., et al. (2020). BIP-340/341/342: Taproot Signature and Scripting.

[2] O'Connor, R. (2017/2024). Simplicity: A New Language for Blockchains and Unified Verification.

[3] Back, A., et al. (2025). Simplicity-Unchained: Practical Covenants on Bitcoin L1.

[4] Hanson, R. (2002). Scoring Rules for Prediction Markets.

[5] LOL. (2025). OPI-001: swap Operation for Universal BRC-20 Extension.

[6] LOL. (2025). OPI-2: The Curve Extension for Trustless Programmatic Token Distribution.

[7] Orlovsky, M., & Zucco, G. (2023). RGB: A Scalable & Confidential Smart Contracts System for Bitcoin.

[8] Osuntokun, O. (2022). Taproot Assets (formerly Taro).

[9] Sztorc, P. (2015). Truthcoin: Peer-to-Peer Oracle System and Prediction Marketplace.

[10] Lau, J. (2016). BIP-114: Merkelized Abstract Syntax Tree (MAST).

[11] Zkao, et al. (2022). UTXOracle: Deriving Fiat Price from Bitcoin UTXO Entropy.

[12] Chepurnoy, A., et al. (2018). Self-Reproducing Coins as Universal Turing Machine.

[13] Linus, R. (2024). Binohash: Transaction Introspection Without Softforks.